

**Laporan Praktikum  
Pemrograman Web 2  
Pertemuan 6  
CRUD**



Nama Mahasiswa: Tegar Raditya Hikmawan

NIM:24/535872/SV/24321

Kelas: B2

Dosen Pengampu: Dinar Nugroho Pratomo, S.Kom., M.IM., M.Cs.

**Sekolah Vokasi  
Universitas Gadjah Mada  
Yogyakarta  
2025**

# Daftar Isi

|                                    |           |
|------------------------------------|-----------|
| <b>Bab 1. Pendahuluan</b>          | <b>2</b>  |
| 1.1 Latar Belakang . . . . .       | 2         |
| 1.2 Tujuan Praktikum . . . . .     | 2         |
| <b>Bab 2. Dasar Teori</b>          | <b>3</b>  |
| 2.1 Pengenalan Laravel . . . . .   | 3         |
| 2.2 MVC pada Laravel . . . . .     | 3         |
| 2.3 CRUD . . . . .                 | 4         |
| <b>Bab 3. Hasil dan Pembahasan</b> | <b>6</b>  |
| 3.1 Setup . . . . .                | 6         |
| 3.2 Create . . . . .               | 10        |
| 3.3 Update . . . . .               | 13        |
| 3.4 Delete . . . . .               | 17        |
| <b>Bab 4. Kesimpulan dan Saran</b> | <b>20</b> |
| 4.1 Kesimpulan . . . . .           | 20        |

# Bab 1. Pendahuluan

## 1.1 Latar Belakang

Pemrograman web saat ini menjadi salah satu keterampilan penting dalam bidang teknologi informasi. Aplikasi berbasis web semakin banyak digunakan, sehingga diperlukan kemampuan untuk membangun aplikasi yang tidak hanya berjalan dengan baik, tetapi juga mudah dikembangkan dan dikelola.

Laravel adalah salah satu framework PHP yang banyak digunakan karena menyediakan berbagai fitur yang memudahkan pengembangan, seperti sistem routing, template Blade, dan pengelolaan database dengan Eloquent. Dengan menggunakan Laravel, pengembang dapat membuat aplikasi web secara lebih cepat, terstruktur, dan aman.

Praktikum ini bertujuan untuk memberikan pemahaman kepada mahasiswa mengenai pemrograman web lanjutan menggunakan Laravel. Melalui latihan langsung, mahasiswa akan belajar membangun aplikasi web dinamis dengan konsep-konsep penting seperti autentikasi, manajemen data, dan penerapan pola MVC. Dengan demikian, mahasiswa diharapkan memiliki dasar yang kuat untuk mengembangkan aplikasi web modern.

## 1.2 Tujuan Praktikum

1. Mahasiswa mampu mengenali cara penggunaan konsep MVC
2. Mahasiswa mengetahui cara membuat CRUD
3. Mahasiswa dapat menggunakan / menerapkan CRUD pada project yang dibuat

## Bab 2. Dasar Teori

### 2.1 Pengenalan Laravel

Laravel adalah salah satu framework PHP paling populer dan banyak digunakan di dunia pengembangan web. Dibuat oleh Taylor Otwell, Laravel dirancang untuk membuat proses pengembangan aplikasi web menjadi lebih mudah, lebih cepat, dan lebih menyenangkan. Dengan pendekatan yang elegan dan syntax yang mudah dibaca, Laravel menyediakan alat dan fitur yang kuat untuk membantu pembangunan aplikasi dari yang sederhana hingga kompleks. [1]

Keunggulan Laravel antara lain[1]:

- Eloquent ORM : Eloquent menyederhanakan interaksi dengan database dengan cara yang lebih intuitif dan object-oriented
- Migrations : Kemudahan dalam mengelola struktur database aplikasi
- Blade Template Engines : Fitur untuk menyaring HTTP request yang masuk ke aplikasi
- Middleware : Fitur untuk menyaring HTTP request yang masuk ke aplikasi
- Routing yang mudah : Sistem routing yang sederhana namun fleksibel
- Autentikasi : Sistem otentikasi yang lengkap dan mudah dikonfigurasi
- Laravel Mix : Tool untuk mengelola asset dan melakukan compiling

### 2.2 MVC pada Laravel

Arsitektur aplikasi Laravel dirancang untuk menjadi intuitif dan fleksibel[1]. Salah satu prinsip dan pola desain software yang diadopsi Laravel adalah Model View Controller [1]. Model View Controller atau yang dapat disingkat MVC adalah sebuah pola arsitektur dalam membuat sebuah aplikasi dengan cara memisahkan kode menjadi tiga bagian yang terdiri dari:

- Model

Bagian yang bertugas untuk menyiapkan, mengatur, memanipulasi, dan mengorganisasikan data yang ada di database. [2]

- View

Bagian yang bertugas untuk menampilkan informasi dalam bentuk Graphical User Interface (GUI). [2]

- Controller

Bagian yang bertugas untuk menghubungkan serta mengatur model dan view agar dapat saling terhubung. [2]

Dalam prosesnya, MVC memiliki alur:

- Proses pertama adalah view akan meminta data untuk ditampilkan dalam bentuk grafis kepada pengguna. [2]
- Permintaan tersebut diterima oleh controller dan diteruskan ke model untuk diproses. [2]
- Model akan mencari dan mengolah data yang diminta di dalam database. [2]
- Setelah data ditemukan dan diolah, model akan mengirimkan data tersebut kepada controller untuk ditampilkan di view. [2]
- Controller akan mengambil data hasil pengolahan model dan mengaturnya di bagian view untuk ditampilkan kepada pengguna. [2]

## 2.3 CRUD

CRUD adalah serangkaian fungsionalitas mendasar yang biasanya digunakan dalam manajemen database. CRUD mencakup fungsi Create, Read, Update, dan Delete. Fungsi tersebut merupakan dasar untuk sebagian besar aplikasi software yang berhubungan dengan penyimpanan dan pengambilan data. CRUD merupakan fondasi interaksi dengan database dan sering diimplementasikan dalam logika backend aplikasi web, software, mobile app, dan sistem lain yang memerlukan manajemen data. Dengan menyediakan fungsionalitas CRUD, developer dapat secara efektif mengelola dan memanipulasi data dalam aplikasi mereka. [3]

Fungsi CRUD antara lain:

- Create

Dengan fungsi Create, pengguna dapat menambahkan catatan atau entri baru ke database. Fungsi ini meliputi memasukkan data ke dalam database, membuat baris baru dalam tabel, dan menentukan nilai untuk atribut (kolom) dari baris itu. Sebagai contoh, saat pengguna membuat akun pengguna baru di website, informasi yang relevan seperti nama, alamat e-mail, dan password dimasukkan ke dalam database. [3]

- Read

Fungsi Read membantu pengguna untuk mengambil atau mengakses data yang ada dari database, seperti saat mencari catatan tertentu atau memperoleh semua catatan yang memenuhi kriteria tertentu. Misalnya, saat pengguna masuk ke website, aplikasi akan melakukan operasi Read untuk mengakses informasi akun mereka dari database dan menampilkannya di layar. [3]

- Update

Update memungkinkan pengguna untuk mengubah atau mengedit data yang ada di database. Misalnya, jika pengguna ingin mengubah alamat e-mail-nya atau memperbarui informasi tertentu di profilnya, aplikasi akan menjalankan operasi Update untuk mengubah data yang relevan di database. [3]

- Delete

Fungsi Delete memberi akses ke pengguna untuk menghapus data yang tidak diinginkan dari database. Dengan demikian, saat pengguna ingin menghapus akunnya di suatu website, aplikasi akan melakukan Delete untuk menghapus data tersebut dari database. [3]

## Bab 3. Hasil dan Pembahasan

Pada praktikum kali ini, diberikan tugas untuk kita melatih kemampuan kita dalam memanfaatkan CRUD dengan fokus pada Create, Update, dan Delete. Untuk menambahkan setiap fungsi CRUD dalam web kita, maka pertama kita perlu membuat tabel dalam database kita. Setelahnya, kita bisa menghubungkan table tersebut dengan model. Model yang telah dibuat nanti bisa dipanggil oleh controller. Terakhir untuk menentukan view mana yang akan menjalankan fungsi apa, kita bisa menentukannya melalui route. Karena kita telah menampilkan tabel dari pertemuan sebelumnya, berikut adalah kode migration, model, controller, route, dan view untuk menampilkan semua data table dari database.

### 3.1 Setup

#### 1. Migration

```
1 public function up(): void
2 {
3     Schema::create('books', function (Blueprint $table
4         ) {
5         $table->id();
6         $table->string('judul');
7         $table->integer('harga');
8         $table->string('penulis');
9         $table->date('tgl_terbit');
10        $table->timestamps();
11    });
12 }
```

#### 2. Model

```
1 class Buku extends Model
2 {
3     use HasFactory;
4
5     protected $table = 'books';
6     protected $fillable = ['judul', 'penulis', 'harga',
7         , 'tgl_terbit'];
8
9     protected $casts = [
10         'tgl_terbit' => 'datetime',
11     ];
12 }
```

## 3. Controller

```
1 class BukuController extends Controller
2 {
3     public function search(Request $request)
4     {
5         $search      = $request->input('judul');
6         $nama_penulis = $request->input('penulis');
7
8         $query = Buku::query();
9
10        if (!empty($search)) {
11            $query->where('judul', 'LIKE', '%' .
12                $search . '%');
13        }
14
15        if (!empty($nama_penulis)) {
16            $query->where('penulis', $nama_penulis);
17        }
18
19        return $query->get();
20    }
21    public function getStats()
22    {
23        return [
24            'five_newest' => Buku::orderBy('tgl_terbit', 'desc')->take(5)->get(),
25            'total_books' => Buku::count(),
26            'total_price' => Buku::sum('harga'),
27            'min_price'   => Buku::min('harga'),
28            'max_price'   => Buku::max('harga'),
29        ];
30    }
31    public function index(Request $request)
32    {
33        $data_buku = $this->search($request);
34        $stats     = $this->getStats();
35
36        return view('buku.index', array_merge(
37            $stats,
38            [
39                'data_buku'      => $data_buku,
40                'search'         => $request->input('
41                    judul'),
42                'nama_penulis' => $request->input('
43                    penulis'),
44            ]
45        ));
46    }
47 }
```



## 4. Route

```
1 Route::get('/buku', [BukuController::class, 'index'])
   ->name('buku');
```

## 5. View

```
1 @extends("layouts.master")
2
3 @section('title')
4 Buku
5 @endsection
6
7 @section('content')
8 <div class="text-white text-center">
9     <h1 class="">Daftar Buku</h1>
10
11     <div class="mt-5 d-flex gap-3 justify-content-
12         between">
13         <div class="card mb-3" style="width: 18rem;">
14             <div class="card-body">
15                 <h5 class="card-title">Total Buku</h5>
16                 <p class="card-text">{{$total_books}}</p>
17             </div>
18         </div>
19         <div class="card mb-3" style="width: 18rem;">
20             <div class="card-body">
21                 <h5 class="card-title">Total Harga</h5>
22                 <p class="card-text">{{"Rp." .
23                     number_format((float)$total_price, 2,
24                     ',', '.')}}</p>
25             </div>
26         </div>
27         <div class="card mb-3" style="width: 18rem;">
28             <div class="card-body">
29                 <h5 class="card-title">Harga minimal</h5>
30                 <p class="card-text">{{"Rp." .
31                     number_format((float)$min_price, 2,
32                     ',', '.')}}</p>
33             </div>
34         </div>
35         <div class="card mb-3" style="width: 18rem;">
36             <div class="card-body">
37                 <h5 class="card-title">Haga maksimal</h5>
38                 <p class="card-text">{{"Rp." .
39                     number_format((float)$max_price, 2,
40                     ',', '.')}}</p>
41             </div>
42         </div>
43     </div>
44 </div>
```

```

34         </div>
35     </div>
36 </div>
37
38 <h3 class="text-start mt-3">5 Judul paling baru</h3>
39 <div class="d-flex align-item-start">
40     @foreach ($five_newest as $index=>$buku)
41         <div class="card mx-3" style="width: 18rem;">
42             <div class="card-title">
43                 
46                 <h5>{{ $buku->judul }}</h5>
47             </div>
48             <div class="card-body">
49                 <p class="card-text">
50                     <b>{{ $buku->penulis }}</b>
51                     <div>{{ $buku->tgl_terbit->format('
52                         d/m/Y')}}</div>
53                 </p>
54             </div>
55         </div>
56     @endforeach
57 </div>
58
59 <form method="GET" class="mb-3 mt-5 d-flex gap-2">
60     <input type="text" name="judul" class="form-
61         control"
62         placeholder="Cari judul buku" value="{{
63             $search }}">
64
65     <select class="form-select" aria-label="
66         Default select example" name="penulis">
67         <option disabled {{ empty($nama_penulis) ?
68             'selected' : '' }}>Cari Penulis</
69         option>
70         @foreach ($data_buku as $penulis)
71             <option value="{{ $penulis->penulis }}"
72                 {{ $nama_penulis == $penulis->
73                     penulis ? 'selected' : '' }}>
74                 {{ $penulis->penulis }}
75             </option>
76         @endforeach
77     </select>
78
79     <button type="submit" class="btn btn-primary">
80         Cari</button>
81     @if (!empty($search) || !empty($nama_penulis))

```

```

72         <a href="{ url('/buku') }}" class="btn
           btn-secondary">X</a>
73     @endif
74 </form>
75
76 <table class="table table-striped table-light">
77     <thead>
78         <tr>
79             <th>id</th>
80             <th>Judul Buku</th>
81             <th>Penulis</th>
82             <th>Harga</th>
83             <th>Tanggal Terbit</th>
84             <th>Aksi</th>
85         </tr>
86     </thead>
87     <tbody>
88         @foreach ($data_buku as $index => $buku)
89             <tr>
90                 <td>{{ $index+1 }}</td>
91                 <td>{{ $buku->judul }}</td>
92                 <td>{{ $buku->penulis }}</td>
93                 <td>{{ "Rp.".number_format((float)
                    $buku->harga, 2, ',', '.') }}</td>
94                 <td>{{ $buku->tgl_terbit->format('d/m/
                    Y') }}</td>
95                 <td class=""></td>
96             </tr>
97         @endforeach
98     </tbody>
99 </table>
100 </div>
101 @endsection

```

Setelahnya, kita tidak perlu membuat migration dan model baru karena akan menggunakan tabel yang sama.

## 3.2 Create

### 1. Controller

```

1 public function create()
2 {
3     return view('buku.create');
4 }
5 public function store(Request $request)
6 {
7     $validated = $request->validate([
8         'judul' => 'required|max:255',
9         'penulis' => 'required|max:45',

```

```

10         'harga' => 'required|integer',
11         'tgl_terbit' => 'required|date'
12     ]);
13
14     Buku::create($validated);
15     return redirect()->route('buku')->with('success',
16         'A new book has been added!!');
17 }

```

Karena kita akan menambahkan item pada page yang berbeda, maka kita perlu dua method, pertama untuk memindahkan halaman, dan kedua untuk menambahkan data pada database. Method create akan mereturn halaman view untuk create buku. Lalu setelah user mengisi setiap value pada kolom, maka akan menggunakan method store yang mana akan menerima parameter Request dari user. Request yang diterima lalu divalidasi. Setelah itu, memanggil model Buku dengan function create() untuk menambahkannya ke database. Terakhir, method store memiliki return untuk me-redirect ke halaman seelumnya.

## 2. Route

```

1 Route::get('/buku/create', [BukuController::class, '
    create'])->name('buku.create');
2 Route::post('/buku', [BukuController::class, 'store'])
    ->name('buku.store');

```

Karena kita memiliki dua method, maka kita juga membuat dua route tambahan. Yang pertama adalah route untuk '/buku/create' dan akan menjalankan method create. Kedua adalah untuk '/buku' dan akan menjalankan method store. Bedanya dengan route '/buku' yang menjalankan method index adalah http methodnya. Route '/buku' ini menggunakan method post yang mana untuk mengirim data ke database.

## 3. View

Button untuk mengarah ke '/buku/create'

```

1 <a href="{{route('buku.create')}}" class="btn btn-
    warning">+Tambah</a>

```

Form pada '/buku/create'

```

1 <form method="POST" action="{{route('buku.store')}}"
    class="text-start mt-3 bg-success p-3 bg-opacity-25
    ">
2     @csrf
3     <div class="mb-3">
4         <label class="form-label">Judul</label>
5         <input type="text" class="form-control" name="
            judul">
6     </div>
7     <div class="mb-3">
8         <label class="form-label">Nama penulis</label>
9         <input type="text" class="form-control" name="
            penulis">

```

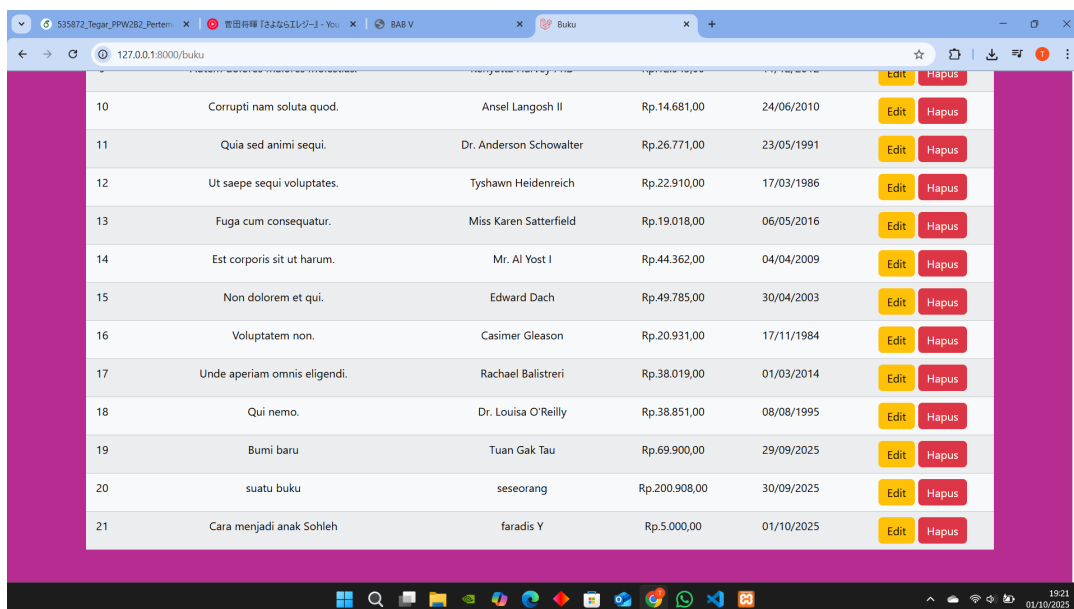
```

10     </div>
11     <div class="mb-3">
12         <label class="form-label">Harga</label>
13         <input type="number" class="form-control" name
            ="harga">
14     </div>
15     <div class="mb-5">
16         <label class="form-label">Tanggal terbit</
            label>
17         <input type="date" class="form-control" name="
            tgl_terbit">
18     </div>
19
20     <button type="submit" class="btn btn-primary">
        Submit</button>
21     <a href="{{route('buku')}}" class="btn btn-
        secondary">Kembali </a>
22 </form>

```

Form ini menggunakan method "POST" untuk mengirim data, dan action yang menuju "buku.store". Tidak lupa, laravel perlu untuk menambahkan @csrf untuk pengisian form. Setiap input field dibuat untuk menerima setiap value kolom table 'books'.

+
+Tambah



### 3.3 Update

#### 1. Controller

```

1 public function edit($id)
2 {
3     $get_buku = Buku::find($id);
4     return view('buku.edit', compact('get_buku'));
5 }
6 public function update(Request $request, $id)
7 {
8     $validated = $request->validate([
9         'judul'      => 'required|max:255',
10        'penulis'     => 'required|max:45',
11        'harga'       => 'required|integer',
12        'tgl_terbit'  => 'required|date'

```

```

13     });
14
15     $buku = Buku::findOrFail($id);
16     $buku->update($validated);
17
18     return redirect()->route('buku.edit', $id)->with('
19         success', 'The book has been updated
        successfully!');
    }

```

Sama halnya dengan create, kita memerlukan dua method untuk berpindah halaman dan handle fungsi update-nya. Method edit menerima parameter \$id yang akan digunakan untuk mendapatkan item yang ingin di-update. Setelah didapat, maka akan return view '/buku/edit' dengan setiap value kolom dalam baris dengan id yang diberikan. Dalam method update, akan menerima Request dan id. Request yang didapat akan divalidate. Setelah divalidate, maka menggunakan id yang diberi untuk mencari item dengan id yang sama. Setelah ditemukan, maka digunakan function update() untuk mengupdate item. Return dari method ini adalah me-redirect ke halaman sebelumnya.

## 2. Route

```

1 Route::get('/buku/edit/{id}', [BukuController::class,
    'edit'])->name('buku.edit');
2 Route::put('/buku/edit/{id}', [BukuController::class,
    'update'])->name('buku.update');

```

Karena kita memiliki dua method, maka kita juga membuat dua route tambahan. Yang pertama adalah route untuk '/buku/edit/id' yang menerima parameter id dan akan menjalankan method edit. Kedua adalah untuk route sama yaitu '/buku/edit/id' dan akan menjalankan method update. Beda antara route '/buku/edit/id' yang menjalankan method edit dengan yang menjalankan method update adalah http methodnya. Route dengan method edit menggunakan http method get yang mana untuk mendapat data dari database. Sedangkan route dengan method update menggunakan http method put yang digunakan khusus untuk mengupdate suatu item.

## 3. View

Button untuk mengarah ke '/buku/edit/id'

```

1 <td>
2     <a type="button" href="{{route('buku.edit', $buku
3         ->id)}}" class="btn btn-warning">
4         Edit
5     </a>
</td>

```

Form pada '/buku/edit/id'

```

1 <form method="POST" action="{{route('buku.update',
    $get_buku->id)}}" class="text-start mt-3 bg-success
    p-3 bg-opacity-25">

```

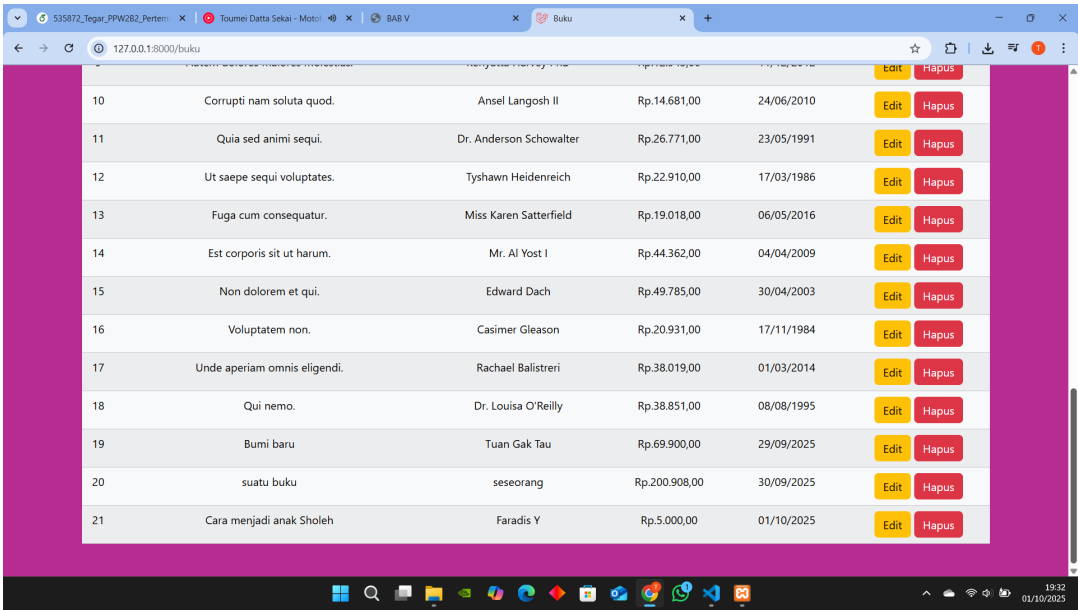
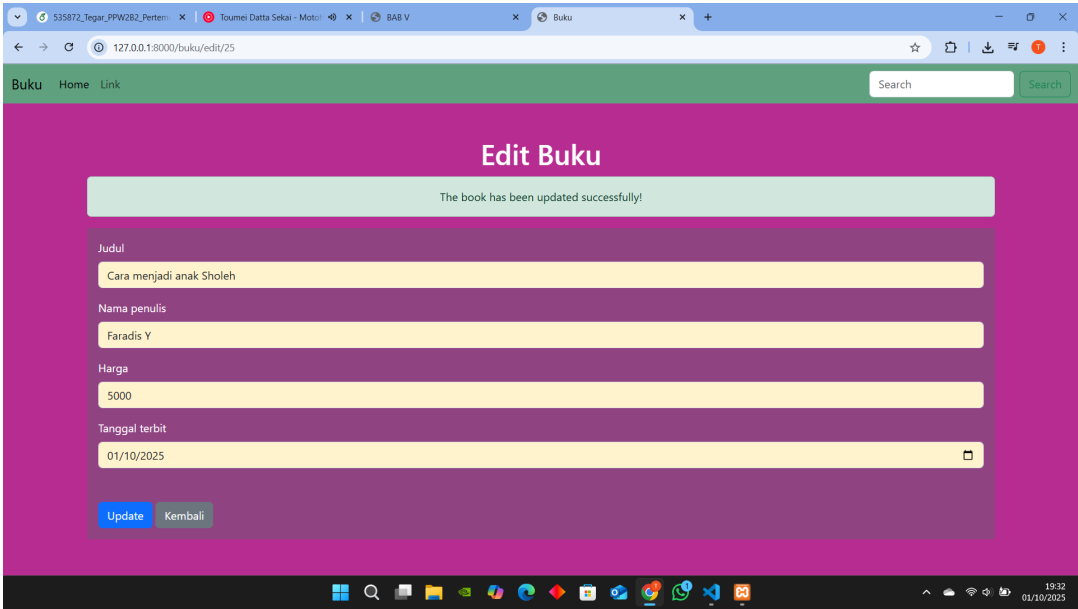
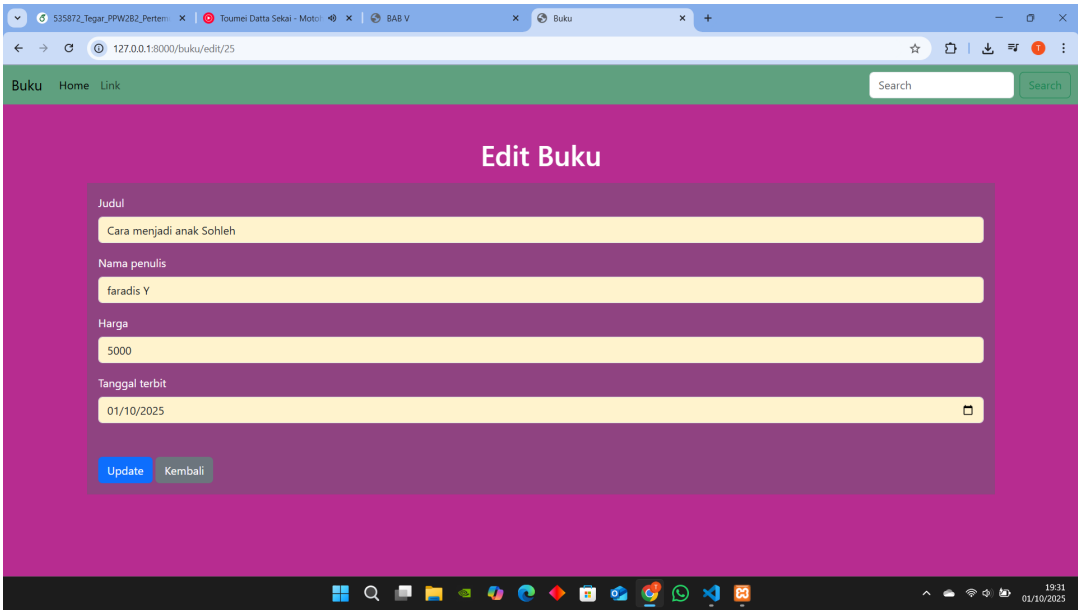
```

2      @csrf
3      @method('PUT')
4
5      <div class="mb-3">
6          <label class="form-label">Judul</label>
7          <input type="text" class="form-control bg-
            warning-subtle" name="judul" value="{{
              $get_buku->judul}}">
8      </div>
9      <div class="mb-3">
10         <label class="form-label">Nama penulis</label>
11         <input type="text" class="form-control bg-
            warning-subtle" name="penulis" value="{{
              $get_buku->penulis}}">
12     </div>
13     <div class="mb-3">
14         <label class="form-label">Harga</label>
15         <input type="number" class="form-control bg-
            warning-subtle" name="harga" value="{{
              $get_buku->harga}}">
16     </div>
17     <div class="mb-5">
18         <label class="form-label">Tanggal terbit</
            label>
19         <input type="date" class="form-control bg-
            warning-subtle" name="tgl_terbit" value="{{
              $get_buku->tgl_terbit->format('Y-m-d')}}">
20     </div>
21
22     <button type="submit" class="btn btn-primary">
        Update</button>
23     <a href="{{route('buku')}}" class="btn btn-
        secondary">Kembali</a>
24 </form>

```

Form ini awalnya menggunakan method "POST" untuk mengirim data, dan action yang menuju "buku.update" dengan parameter berupa id dari buku yang diedit pada halaman ini. Setelah menambahkan `@csrf`, kita lalu menambahkan `@method('PUT')` untuk merewrite http method yang dijalankan. Setiap input field telah menampilkan value setiap kolom untuk buku yang sedang diedit ini.





### 3.4 Delete

#### 1. Controller

```

1 public function destroy($id){
2     $buku = Buku::find($id);
3     $buku->delete();
4
5     return redirect()->route('buku');
6 }

```

Untuk melakukan fungsi delete, kita hanya perlu satu method bernama destroy. Method destroy menerima parameter \$id untuk menemukan item yang ingin dihapus. Kita mencarinya dengan function find(). Setelah ditemukan, delete() dan lalu redirect() ke '/buku'.

#### 2. Route

```

1 Route::delete('/buku/{id}', [BukuController::class, '
    destroy'])->name('buku.destroy');

```

URL route ini adalah '/buku' dengan tambahan '/id' untuk mengetahui item mana yang ingin dihapus, dan lalu menghapusnya dengan method destroy. Tidak lupa untuk menggunakan delete sebagai http methodnya yang merupakan http method khusus untuk menghapus item.

#### 3. View

Button untuk menampilkan modal

```

1 <button type="button" class="btn btn-danger" data-bs-
    toggle="modal" data-bs-target="#modal-{{ $buku->id
    }}">
2     Hapus
3 </button>

```

Modal dengan form untuk melakukan delete

```

1 <div class="modal fade" id="modal-{{ $buku->id }}"
    tabindex="-1" aria-labelledby="exampleModalLabel"
    aria-hidden="true">
2     <div class="modal-dialog modal-dialog-centered">
3         <div class="modal-content">
4             <div class="modal-header">
5                 <h1 class="modal-title fs-5" id="
                    exampleModalLabel">Yakin ingin
                    menghapus?</h1>
6                 <button type="button" class="btn-close"
                    data-bs-dismiss="modal" aria-label="
                    Close"></button>
7             </div>
8             <div class="modal-body text-start">
9                 Judul : {{ $buku->judul }} <br>
10                Penulis : {{ $buku->penulis }} <br>

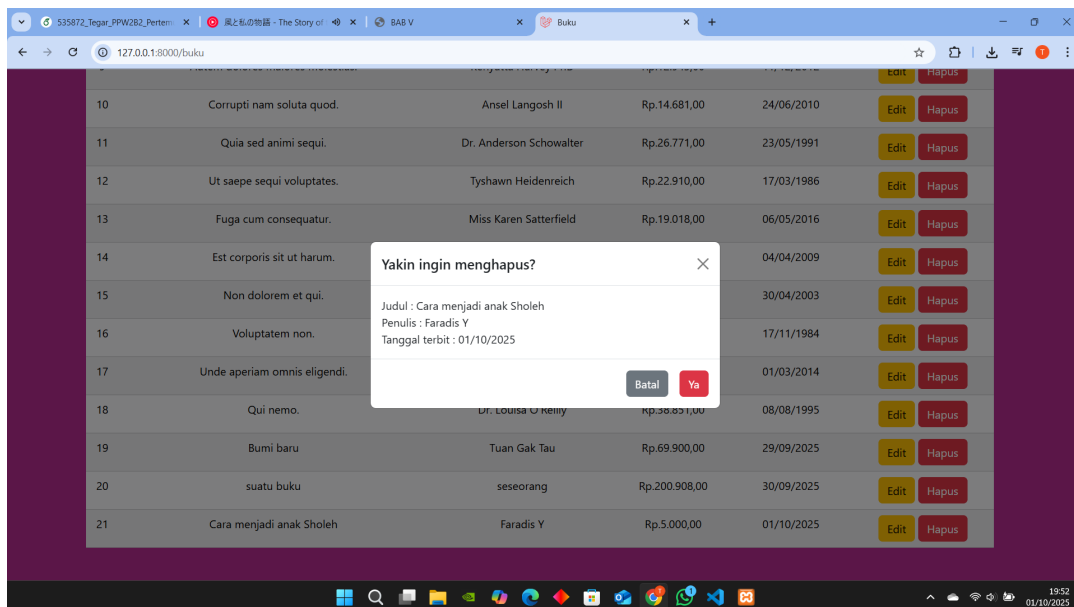
```

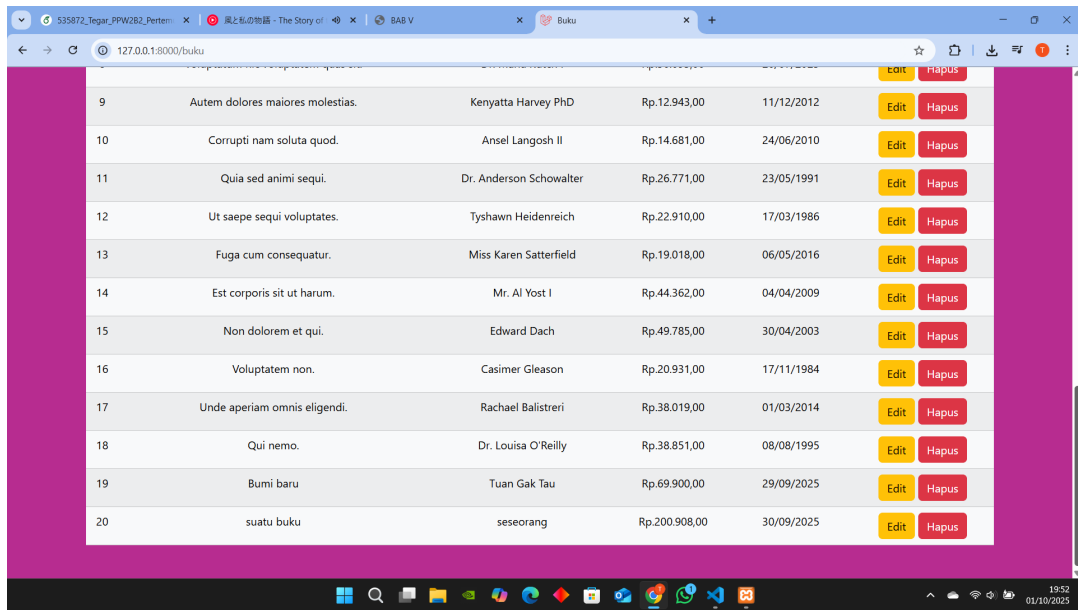
```

11         Tanggal terbit : {{ $buku->tgl_terbit->
            format('d/m/Y') }}
12     </div>
13     <div class="modal-footer">
14         <button type="button" class="btn btn-
            secondary" data-bs-dismiss="modal">
            Batal </button>
15         <form id="deleteForm" action="{{ route('
            buku.destroy', $buku->id) }}" method="
            POST">
16             @csrf
17             @method('DELETE')
18             <button type="submit" class="btn btn-
            danger">Ya </button>
19         </form>
20     </div>
21 </div>
22 </div>
23 </div>

```

Dalam kolom yg sama dengan tombol edit, kita buat tombol button yang memiliki atribut data-bs-target memiliki id yang sesuai dengan buku. Modal yang muncul adalah modal yang memang khusus untuk suatu id buku. Untuk menconfirm hapus, maka kita perlu tombol submit. Jika kita perlu tombol submit, maka kita perlu form dengan method yang telah direwrite dengan `@method('DELETE')` untuk menjalankan http method delete.





|    |                                  |                         |               |            |      |       |
|----|----------------------------------|-------------------------|---------------|------------|------|-------|
| 9  | Autem dolores maiores molestias. | Kenyatta Harvey PhD     | Rp.12.943,00  | 11/12/2012 | Edit | Hapus |
| 10 | Corrupti nam soluta quod.        | Ansel Langosh II        | Rp.14.681,00  | 24/06/2010 | Edit | Hapus |
| 11 | Quia sed animi sequi.            | Dr. Anderson Schowalter | Rp.26.771,00  | 23/05/1991 | Edit | Hapus |
| 12 | Ut saepe sequi voluptates.       | Tyshawn Heidenreich     | Rp.22.910,00  | 17/03/1986 | Edit | Hapus |
| 13 | Fuga cum consequatur.            | Miss Karen Satterfield  | Rp.19.018,00  | 06/05/2016 | Edit | Hapus |
| 14 | Est corporis sit ut harum.       | Mr. Al Yost I           | Rp.44.362,00  | 04/04/2009 | Edit | Hapus |
| 15 | Non dolore et qui.               | Edward Dach             | Rp.49.785,00  | 30/04/2003 | Edit | Hapus |
| 16 | Voluptatem non.                  | Casimer Gleason         | Rp.20.931,00  | 17/11/1984 | Edit | Hapus |
| 17 | Unde aperiam omnis eligendi.     | Rachael Balistreri      | Rp.38.019,00  | 01/03/2014 | Edit | Hapus |
| 18 | Qui nemo.                        | Dr. Louisa O'Reilly     | Rp.38.851,00  | 08/08/1995 | Edit | Hapus |
| 19 | Bumi baru                        | Tuan Gak Tau            | Rp.69.900,00  | 29/09/2025 | Edit | Hapus |
| 20 | suatu buku                       | seseorang               | Rp.200.908,00 | 30/09/2025 | Edit | Hapus |

Link Repository: [Repository Laravel Day 6](#)

## Bab 4. Kesimpulan dan Saran

### 4.1 Kesimpulan

Berdasarkan dasar teori yang telah dipaparkan, dapat disimpulkan beberapa hal penting:

1. Laravel merupakan framework PHP yang populer karena menyediakan berbagai fitur modern seperti Eloquent ORM, Migrations, Blade Template Engine, Middleware, sistem routing, autentikasi, serta Laravel Mix yang mendukung efisiensi dalam pengembangan aplikasi web.
2. Pola arsitektur Model-View-Controller (MVC) yang digunakan Laravel membantu memisahkan data (Model), tampilan (View), dan logika aplikasi (Controller), sehingga struktur kode menjadi lebih terorganisir, mudah dipelihara, dan fleksibel.
3. Model bertugas untuk mengelola data dan interaksi dengan database, View menampilkan data dalam bentuk Graphical User Interface (GUI), sedangkan Controller menjadi penghubung antara Model dan View agar aplikasi dapat berjalan sesuai kebutuhan pengguna.
4. Alur kerja MVC memastikan permintaan pengguna dari View diteruskan ke Controller, diproses oleh Model, kemudian hasilnya dikembalikan lagi melalui Controller untuk ditampilkan di View.
5. Konsep CRUD (Create, Read, Update, Delete) menjadi dasar dalam pengelolaan data pada aplikasi berbasis database. CRUD memungkinkan pengguna menambahkan, membaca, memperbarui, serta menghapus data dengan cara yang konsisten dan sistematis.
6. Kombinasi antara framework Laravel, penerapan pola MVC, serta implementasi fungsionalitas CRUD memberikan pondasi yang kuat untuk membangun aplikasi web yang terstruktur, efisien, dan mudah dikembangkan baik dalam skala kecil maupun besar.

# Daftar Pustaka

- [1] Kurniawan, A. (2024). *Laravel Workshop: Belajar dengan Membangun Proyek Aplikasi Web*. Ilmu Data Publisher. Diakses dari [https://www.google.co.id/books/edition/Laravel\\_Workshop\\_Belajar\\_dengan\\_Membangu/oczSEAAQBAJ?hl=id&gbpv=1](https://www.google.co.id/books/edition/Laravel_Workshop_Belajar_dengan_Membangu/oczSEAAQBAJ?hl=id&gbpv=1)
- [2] Setiawan, R. (2021). Apa Itu MVC? Pahami Konsepnya dengan Baik. *dicoding*. Diakses dari <https://www.dicoding.com/blog/apa-itu-mvc-pahami-konsepnya/>
- [3] (2025). Apa itu CRUD?. *RevoUpedia*. Diakses dari <https://www.revou.co/id/kosakata/crud>